

Tarea #3

Reto final

PL/SQL

Bases de datos

1º DAW

1. Descripción del reto

Este proyecto tiene como objetivo que demuestres que eres capaz de diseñar, implementar y documentar una base de datos completa para un caso de uso real. No se trata solo de que el código funcione: se trata de que cada decisión tenga sentido para el negocio que hayas elegido y de que puedas explicar por qué la has tomado.

Trabajarás de forma individual. A lo largo del proyecto aplicarás todo lo visto en clase: modelado relacional, normalización, SQL y PL/SQL.

2. Elige tu caso de uso

El primer paso es tuyo. Elige una empresa o negocio real (o realista) que necesite una base de datos. Algunos ejemplos orientativos:

- Clínica veterinaria
- Academia de idiomas con reserva de clases
- Tienda online con gestión de stock y pedidos
- Gimnasio con socios, tarifas y clases
- Biblioteca con préstamos y reservas
- Restaurante con carta, reservas y comandas
- Sistema de turnos para un centro de salud

No se admiten casos de uso genéricos o inventados sin coherencia. El negocio que elijas debe tener suficiente complejidad como para justificar varias tablas, relaciones reales y reglas de negocio que puedan automatizarse con triggers.

Criterio clave: cualquiera que lea tu trabajo debe entender por qué cada decisión tiene sentido para ese negocio concreto. Si eliges algo, debes conocerlo lo suficiente como para modelarlo bien.

3. Fases del proyecto

3.1 Fase 1 - Análisis y modelo relacional:

Identifica las entidades del negocio, sus atributos y las relaciones entre ellas. A partir de ahí:

- Dibuja el diagrama Entidad-Relación (E/R).
- Tradúcelo al modelo relacional.
- Normaliza todas las tablas.
- Justifica cada decisión: por qué esa clave primaria, por qué esa relación N:M, por qué normalizas de esa forma.

3.2 Fase 2 - Creación de tablas y datos de ejemplo:

Crea las tablas en PL/SQL incluyendo todas las restricciones necesarias:

- PRIMARY KEY, FOREIGN KEY, NOT NULL, CHECK, UNIQUE.
- Inserta datos de ejemplo realistas: mínimo 5-10 filas por tabla principal.
- Los datos deben ser suficientes para que las consultas posteriores devuelvan resultados significativos.

3.3 Fase 3 - Batería de consultas SQL:

Crea un conjunto de 10 consultas que un usuario real de ese negocio podría necesitar.

Deben incluir:

- SELECT con JOIN.
- GROUP BY y HAVING con funciones de agregación (COUNT, SUM, AVG, MAX, MIN).
- Subconsultas.
- Cada consulta debe tener un comentario que explique para qué sirve en el contexto del negocio.

3.4 Fase 4 - PL/SQL: procedimientos, funciones y cursores:

Implementa como mínimo:

- 2 procedimientos almacenados con parámetros IN/OUT que realicen operaciones útiles para el negocio.
- 2 funciones que devuelvan un valor calculado con sentido en el contexto del negocio.
- 1 cursor explícito que recorra registros y ejecute lógica sobre ellos.
- Bucles (LOOP, FOR, WHILE) e IF/CASE donde tengan sentido real.

3.5 Fase 5 – Triggers:

Crea 6 triggers que automaticen reglas de negocio reales. Algunos ejemplos de lo que pueden hacer:

- Control y actualización automática de stock.
- Auditoría de cambios en tablas críticas.
- Validación de datos que no puede hacerse con CHECK.
- Cálculo automático de totales o campos derivados.
- Cada trigger debe ir acompañado de una justificación: por qué existe y qué problema de negocio resuelve.

4. Entregables

Debes entregar todos los ficheros comprimidos en un único .zip con tu nombre (Ej.: Miguel_Aladren.zip). Los ficheros .sql deben poder ejecutarse en orden sin errores sobre una base de datos Oracle limpia.

Fichero	Contenido
crear.sql	Creación de tablas, secuencias y restricciones. Incluye los DROP TABLE correspondientes al inicio para facilitar la re-ejecución.
insertar.sql	Datos de ejemplo suficientes para que todas las consultas devuelvan resultados.
consultas.sql	Batería de consultas SQL comentadas, organizadas por temática.
plsql.sql	Procedimientos almacenados, funciones, cursores y bloques PL/SQL.
triggers.sql	Todos los triggers con comentarios que justifiquen su existencia.
Memoria PDF	Documento con la descripción del caso de uso, justificación de las decisiones de diseño y explicación de cada elemento PL/SQL implementado. Debe incluir el diagrama Entidad-Relación y el modelo relacional.

5. Rúbrica de evaluación

La nota final se distribuye de la siguiente forma. Recuerda que la justificación de las decisiones tiene un peso real: no basta con que el código funcione.

Criterio	Puntuación
Diseño del modelo E/R y normalización	2 puntos
Creación de tablas con restricciones y datos de ejemplo coherentes	1 punto
Batería de consultas SQL (variedad, complejidad y comentarios)	2 puntos
Procedimientos, funciones y cursores PL/SQL	2 puntos
Triggers con justificación de negocio	2 puntos
Calidad de los comentarios y de la memoria justificativa	1 punto
TOTAL	10 puntos

6. Criterios de calidad

Se espera que tengas en cuenta lo siguiente:

- Coherencia entre el negocio elegido y el diseño de la base de datos.
- Justificación. Capacidad de explicar y defender las decisiones tomadas.
- Legibilidad del código SQL y PL/SQL: nombres descriptivos, indentación consistente y comentarios útiles (no obvios).
- Datos de ejemplo. Que los datos de ejemplo sean realistas y permitan probar los casos límite.
- Aplicabilidad. Que los triggers y procedimientos resuelvan problemas reales del negocio, que no sean artificiales.